Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2
По дисциплине: «КиСБ»
Тема: «Избыточное кодирование данных
Код Хемминга и итеративные коды»

Выполнил:
Студент 4-го курса
Группы АС-66
Цеван К. А.
Проверил:
Хацкевич А. С.

Брест 2026

Цель работы

Приобрести практические навыки кодирования и декодирования двоичной информации кодом Хемминга и итеративными кодами, исследовать обнаружение и исправление ошибок.

Ход работы

Вариант 12

1. Код Хемминга

Требуется прочитать текстовое сообщение из файла, представить его в двоичном виде, построить проверочную матрицу, вычислить проверочные символы и передать кодовое слово с 0, 1 и 2 ошибками в случайных позициях. По синдрому необходимо исправить одиночную ошибку и проанализировать двойную. Реализованы коды с минимальным расстоянием 3 и 4.

В файле message.txt записано K12 без перевода строки. В двоичном виде это 01001011 00110001 00110010, то есть k = 24 бита. Для английских букв и цифр байты UTF-8 совпадают с ASCII. Программа обрабатывает байты файла, поэтому также принимает русские буквы в UTF-8, цифры и служебные символы. Допустимая длина — от 2 до 256 байтов; длина в битах вычисляется при чтении.

Число основных проверок r выбирается минимальным из условия $2^r \geq k + r + 1$. При k = 24 получаем r = 5 для d_min = 3, n = 29. При d_min = 4 добавляется проверка общей чётности: r = 6, n = 30.

2. Итеративные коды

Во второй части используется вариант 2 таблицы 5.2 методички. Длина информационного слова k = 20. Рассматриваются все раскладки и количества групп паритетов из этого варианта.

k_1	k_2	z	Группы паритетов
4	5	—	2; 3
2	10	—	2; 3
2	5	2	2; 3; 4; 5
2	2	5	2; 3; 4; 5

1.1 Построение кода Хемминга

Проверочная матрица имеет вид H = [P′ | I]. Столбцы P′ заполняются двоичными векторами веса 2, затем веса 3 и далее в порядке возрастания числа, как в листингах 4.1–4.3 методички. Последние r столбцов образуют единичную матрицу.

```
d_min=3; k=24; r=5; n=29
00000011110000111111110111 | 10000
00011100010111000111111011 | 01000
01100100101011011001110 1 | 00100
10101001001101101010111 0 | 00010
11010010001110110100111 1 | 00001
```

Для расширенного кода к исходной матрице добавляется строка общей чётности и новый столбец. Чтобы вернуть последние столбцы к единичной матрице, последняя строка заменяется XOR всех строк, как указано после формулы 4.8. Получается каноническая матрица:

```
d_min=4; k=24; r=6; n=30
000000111100001111110111 | 100000
000111000101110001111011 | 010000
011001001010110110011101 | 001000
101010010011011010101110 | 000100
110100100011101101001111 | 000010
111111111100000000001111 | 000001
```

Проверочные биты X_r вычисляются как XOR столбцов P′, соответствующих единичным информационным битам. Кодовое слово X_n = (X_k, X_r). Для принятого слова Y_n = (Y_k, Y_r) повторно вычисляются проверки Y_r′ и синдром S = Y_r XOR Y_r′.

При S = 0 ошибки не обнаружены. Для d_min = 3 ненулевой синдром сравнивается со столбцами H и указывает предполагаемую одиночную ошибку. Двойная ошибка может дать синдром другого столбца и привести к неверному исправлению. Для d_min = 4 ненулевой синдром чётного веса означает двойную ошибку; такая ошибка обнаруживается, но не исправляется.

```
X_k=010010110011000100110010
X_r (d=3)=01000
X_n (d=3)=0100101100110001001100 1001000
X_r (d=4)=010001
X_n (d=4)=0100101100110001001100 1 0010001
```

1.2 Передача и исправление ошибок

Позиции ошибок выбираются случайно без повторения. Для воспроизводимости используется seed = 12. Нумерация позиций начинается с 1. Внесённые ошибки известны только эксперименту; декодер получает исключительно принятое слово и проверочную матрицу.

d_min	Ошибок	S	Решение декодера	Y_n′ = X_n
3	0	00000	ошибок не обнаружено	да
3	1	10110	одиночная ошибка исправлена	да
3	2	00011	одиночная ошибка исправлена	нет
4	0	000000	ошибок не обнаружено	да
4	1	000100	одиночная ошибка исправлена	да
4	2	001010	ошибка обнаружена, исправление невозможно	нет

Полный пример для расширенного кода

```
Ошибок: 0
Y_n=01001011001100010011001001000l
Y_r=010001
Y_r'=010001
S=000000
E_n=00000000000000000000000000000000
Y_n'=01001011001100010011001001000l

Ошибок: 1
Y_n=01001011001100010011001001010l
Y_r=010101
Y_r'=010001
S=000100
E_n=00000000000000000000000000000100
Y_n'=01001011001100010011001001000l

Ошибок: 2
Y_n=01000011001100010011101001000l
Y_r=010001
Y_r'=011011
S=001010
E_n=00000000000000000000000000000000
Y_n'=01000011001100010011101001000l
```

Одиночная ошибка исправляется в информационной и в проверочной части слова. При двойной ошибке расширенный код сохраняет принятое слово и сообщает, что обнаружено неисправимое искажение. Совпадение с исходным кодовым словом в таблице — проверка эксперимента, а не дополнительная информация декодера.

1.3 Ключевые фрагменты программы

Каждый столбец H хранится как двоичное число. Поэтому вычисление синдрома сводится к XOR выбранных столбцов, а исправление — к инверсии найденного бита.

```
unsigned syndrome(const unsigned char* word) const {
    unsigned result = 0;
    for (unsigned i = 0; i < n; ++i)
        result ^= columns[i] & (0u - unsigned(word[i]));
    return result;
}

void encode(const unsigned char* data, unsigned char* word) const {
    std::memcpy(word, data, k);
    std::memset(word + k, 0, r);
    unsigned parity = syndrome(word);
    for (unsigned j = 0; j < r; ++j)
        word[k + j] = (parity >> (r - j - 1)) & 1;
}
```

```
unsigned s = syndrome(received);
bool allowed = s != 0;
if (extended) allowed &= (weight(s) & 1u) != 0;
unsigned matches = 0;
for (unsigned j = 0; j < n; ++j)
    matches += unsigned(columns[j] == s);
allowed &= matches == 1;
for (unsigned j = 0; j < n; ++j) {
    error[j] = static_cast<unsigned char>(allowed && columns[j] == s);
    corrected[j] = received[j] ^ error[j];
}
```

Проверены все позиции одиночных ошибок для сообщений длиной 4, 16, 24, 64 и 2048 битов. Для расширенного кода при длинах до 64 битов перебраны все пары позиций двойной ошибки. Одиночные ошибки во всех случаях исправлены, двойные обнаружены без попытки исправления. Для учебного случая k = 4 получена матрица H(7,4), приведённая в методичке.

2.1 Формирование итеративного кода

Информационное слово X_k = 11001010111010001101 вписывается по строкам, сначала в первую плоскость, затем в следующую. Для двумерной раскладки 4 × 5 оно имеет вид:

```
1 1 0 0 1
0 1 0 1 1
1 0 1 0 0
0 1 1 0 1
```

Группы проверок следуют порядку рисунков 5.3–5.4: H — строки; V — столбцы; D — диагонали; A — антидиагонали; Z — одинаковые координаты разных плоскостей. Каждый проверочный символ равен XOR входящих в соответствующую группу информационных битов.

В двумерном случае используются H и V, а при трёх группах добавляется D. Диагонали обычные: для координат строки y и столбца x номер диагонали D равен $x + k_1 - 1 - y$. Всего $k_1 + k_2 - 1$ диагоналей. Включён паритет паритетов X_hv схемы рисунка 5.1.

В трёхмерном случае добавляются группы D, A и Z по порядку. Циклические диагонали на каждой плоскости задаются индексами $(x - y) \bmod k_2$ и $(x + y) \bmod k_2$. По k_2 проверок на плоскость приходится на каждую группу диагоналей. Для Z вычисляется XOR по глубине для каждой пары (y, x). Дополнительный бит сверх групп трёхмерной схемы не добавляется.

Кодовое слово состоит из 20 информационных битов и вычисленных проверок. Ошибки вносятся в случайные различные позиции всего кодового слова, включая проверочные символы. В программе можно задать произвольное 20-битовое слово и кратность ошибок.

Декодирование

Вычисляется синдром — различия между принятыми и пересчитанными проверками. Информационный бит инвертируется, если нарушено строго больше половины его групп проверок. После каждого изменения синдром обновляется. Выбирается бит с наибольшим числом нарушенных проверок; при равенстве — первый. Количество таких шагов ограничено 20. Затем исправляются оставшиеся проверочные символы. Для двумерной схемы дополнительно контролируется общая чётность.

Декодер не знает внесённые позиции и кратность ошибок. Для многократных ошибок метод большинства не гарантирует восстановление, поэтому его работу необходимо оценивать экспериментально.

2.2 Параметры кодов и методика эксперимента

Раскладка	Групп	k	r	n	R = k/n
4×5	2	20	10	30	0,6667
4×5	3	20	18	38	0,5263
2×10	2	20	13	33	0,6061
2×10	3	20	24	44	0,4545
2×5×2	2	20	14	34	0,5882
2×5×2	3	20	24	44	0,4545
2×5×2	4	20	34	54	0,3704
2×5×2	5	20	44	64	0,3125
2×2×5	2	20	20	40	0,5000
2×2×5	3	20	30	50	0,4000
2×2×5	4	20	40	60	0,3333
2×2×5	5	20	44	64	0,3125

Для каждой из 12 конфигураций выполнено по $N_1 = 1000$ испытаний при кратностях i = 0, 1, 2, 3 и 4. Позиции ошибок выбираются равномерно без повторений. Проверяется всё кодовое слово, включая проверочные биты.

N_2 — число испытаний, в которых декодер верно оценил кратность ошибки. N_3 — число испытаний, в которых исправленное слово полностью совпало с переданным X_n. Рассчитываются отношения N_2/N_1 и N_3/N_1. Правильная оценка кратности сама по себе не означает правильного определения позиций.

Во всех конфигурациях безошибочная передача сохранена и все 1000 одиночных ошибок исправлены. Всего выполнено 60 000 испытаний. Отдельная самопроверка также перебирает каждую позицию одиночной ошибки во всех конфигурациях.

Пример ключевого шага декодирования

```
for (unsigned i = 0; i < 20; ++i) {
    unsigned failed = (s & columns[i]).count();
    if (depth == 1) failed -= unsigned(s.bit(r - 1));
    if (2 * failed > groups && failed > best_score) {
        best = int(i);
        best_score = failed;
    }
}
if (best < 0) break;
trial[best] ^= 1;
s = s ^ columns[best];
```

2.3 Результаты исследования

Таблица 1 — Доля верных оценок кратности N_2/N_1. Для i = 0 и i = 1 все значения равны 1.

Раскладка	Групп	i = 2	i = 3	i = 4
4×5	2	0,784	0,466	0,153
4×5	3	0,902	0,797	0,683
2×10	2	0,841	0,594	0,394
2×10	3	0,930	0,871	0,796
2×5×2	2	0,889	0,691	0,471
2×5×2	3	0,982	0,951	0,892
2×5×2	4	1,000	0,986	0,941
2×5×2	5	1,000	1,000	0,997
2×2×5	2	0,919	0,783	0,607
2×2×5	3	1,000	0,994	0,970
2×2×5	4	0,996	0,983	0,956
2×2×5	5	1,000	0,999	0,993

Таблица 2 — Доля полностью восстановленных слов N_3/N_1. Для i = 0 и i = 1 все значения равны 1.

Раскладка	Групп	i = 2	i = 3	i = 4
4×5	2	0,339	0,054	0,001
4×5	3	0,796	0,516	0,212
2×10	2	0,393	0,094	0,029
2×10	3	0,854	0,687	0,475
2×5×2	2	0,677	0,253	0,063
2×5×2	3	0,910	0,764	0,524
2×5×2	4	1,000	0,986	0,939
2×5×2	5	1,000	0,994	0,973
2×2×5	2	0,873	0,623	0,379
2×2×5	3	0,938	0,817	0,656
2×2×5	4	0,990	0,966	0,912
2×2×5	5	1,000	0,992	0,985

2.4 Анализ результатов и запуск

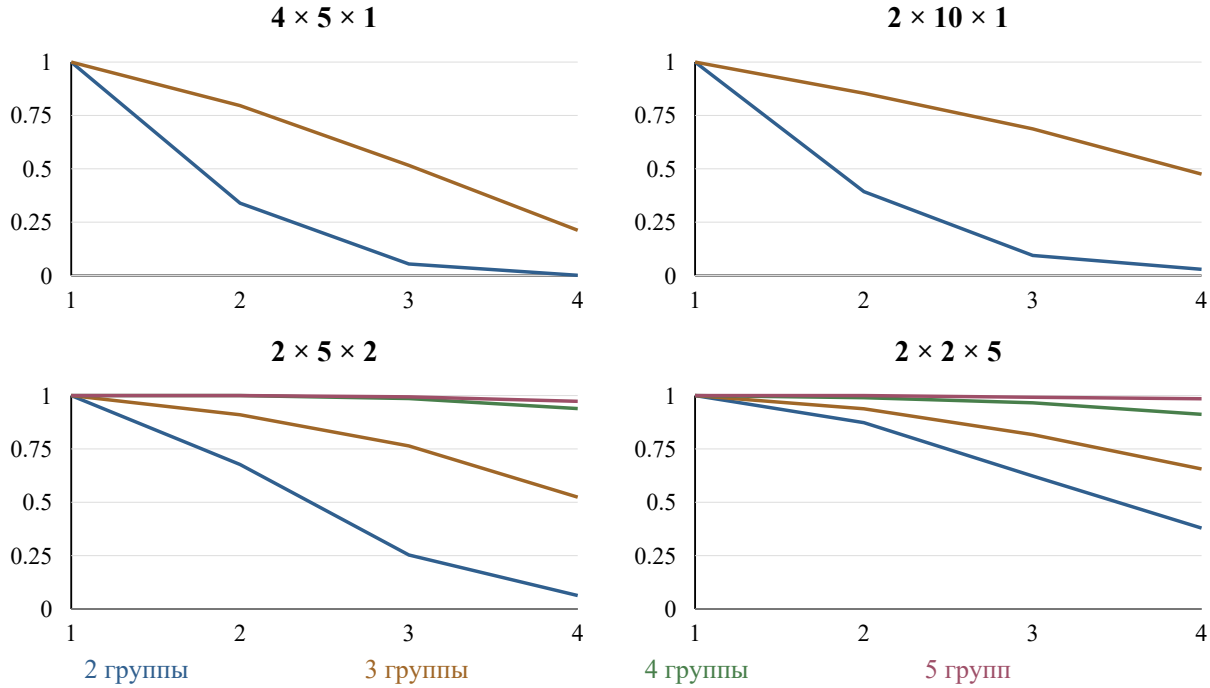


Рисунок 3 — Доля исправленных слов N_3/N_1 в зависимости от кратности ошибки i. Цвет обозначает количество групп паритетов.

При увеличении кратности ошибок доля восстановленных слов уменьшается. Дополнительные проверки обычно улучшают результат, но увеличивают избыточность. Многократные ошибки иногда сохраняют проверки или приводят к другому допустимому слову. Гарантированное исправление всех ошибок такой кратности из эксперимента не следует.

Исходник: lab2/src/tsevan/main.cpp. Из корня репозитория вторая лабораторная собирается и запускается одной командой:

```
.\run_lab2.cmd
```

Файлы сборки находятся в D:/KiSB-extra/build. Сообщение читается из message.txt. Из папки lab2/src/tsevan отдельный запуск с параметрами выполняется так:

```
D:\KiSB-extra\build\lab2.exe message.txt 11001010111010001101 4 1000
```

Аргументы: файл, 20 битов, максимальная кратность и число испытаний. Пример вывода сохранён в sample_output.txt рядом с исходником.

Вывод

Реализован код Хемминга с d_min = 3 и 4. Одиночные ошибки исправляются, двойные обнаруживаются расширенным кодом. Для итеративных кодов выполнены все раскладки варианта 2. С увеличением кратности ошибок доля восстановленных слов снижается; дополнительные проверки улучшают результат ценой увеличения избыточности.